



Pipeline IoT Serverless

Documentation d'implémentation AWS

Devoir Final - Introduction a AWS

ESGIS-TOGO

Master 1 IA-BIG DATA

Juin 2026

Auteur : TCHABODI Sadikou

Responsable : M. Herve LOKOSSOU



Table des matieres

Table des matieres	2
Introduction et contexte du projet.....	4
Architecture mise en place	4
Sous-systeme 1 : Pipeline d'ingestion IoT.....	4
Sous-systeme 2 : Documentation technique (acces securise)	4
Ressources AWS creees par CloudFormation	5
Etape 1 - Preparation de l'environnement	5
1.1 Verifier que l'AWS CLI est installe.....	5
1.2 Configurer vos identifiants AWS	5
1.3 Verifier que la connexion fonctionne	6
1.4 Installer la bibliotheque Python pour les tests	6
Etape 2 - Deploiement de l'infrastructure avec AWS SAM CLI	6
2.1 Verifier les pre-requis SAM et Docker	6
2.2 Valider le template	6
2.3 Construire le package Lambda	6
2.4 Premier deploiement (mode guide interactif)	6
2.5 Suivre la progression du deploiement.....	6
2.6 Verification - Pile CloudFormation CREATE_COMPLETE	7
2.7 Recuperer les URLs generees dans les Outputs	7
Etape 3 - Comprendre le code de la fonction Lambda	8
3.1 Ce que fait la Lambda, etape par etape.....	8
3.2 Gestion des erreurs	8
Etape 4 - Execution des tests avec test_client.py	9
4.1 Test 1 - Payload valide (HTTP 201 attendu)	9
4.2 Test 2 - Records vide (provoque un HTTP 500)	9
4.3 Test 3 - Schema invalide : sensor_id manquant (HTTP 422 attendu)	9
Etape 5 - Verification des donnees dans Amazon S3	9
5.1 Lister les fichiers ingeres via la CLI	9
5.2 Verification depuis la console AWS	9

Etape 6 - Verification des metriques dans Amazon DynamoDB	10
6.1 Scanner la table via la CLI	10
6.2 Verification depuis la console AWS	10
Etape 7 - Deploiement de la documentation technique	11
7.1 Uploader le fichier HTML.....	11
7.2 Tester le blocage de l'accès direct S3	11
7.3 Accéder via CloudFront OAC (accès sécurisé).....	12
Etape 8 - Monitoring et debogage avec Amazon CloudWatch	12
8.1 Accéder aux logs de la Lambda	12
8.2 Log d'une execution réussie (Test 1)	12
8.3 Log d'une execution en echec (Test 2 - ValueError)	13
8.4 Comprendre la difference entre les deux types d'erreurs	13
Reponses aux questions theoriques.....	14
Question 1 - Infrastructure as Code et AWS CloudFormation	14
Question 2 - AWS Lambda et le modele Serverless vs Amazon EC2	14
Question 3 - L'interet d'un CDN CloudFront devant l'API Gateway pour l'IoT	15
Question 4 - S3 comme Data Lake et DynamoDB comme Serving Layer	15
Question 5 - Le modele de responsabilite partagee AWS applique a S3.....	16
Question 6 - Static Website Hosting public vs CloudFront avec OAC	16
Question 7 - Amazon CloudWatch pour superviser et deboguer une Lambda	16
Question 8 - Pourquoi Lambda atteint ses limites avec 50 Go, et quelle alternative	17
Checklist de validation finale	17
Preparation du ZIP pour la soumission	18

Introduction et contexte du projet

Ce projet a pour objectif de mettre en place une architecture cloud entierement serverless capable d'ingerer en temps reel les donnees de capteurs IoT industriels. Concretement, des milliers de capteurs envoient des mesures de temperature, pression et humidite via des requetes HTTP POST.

Notre infrastructure doit recevoir ces donnees, les stocker brutes pour les Data Scientists, et produire des metriques agregees exploitables immediatement. En parallele, une documentation technique du projet est hebergee de facon securisee pour l'equipe interne, accessible uniquement via un CDN avec controle d'accès strict.

Toute cette infrastructure est decrite dans un seul fichier CloudFormation (Infrastructure as Code), ce qui permet de la deployer, modifier ou supprimer en une seule commande.

Architecture mise en place

Le projet se compose de deux sous-systemes independants.

Sous-systeme 1 : Pipeline d'ingestion IoT

Les capteurs envoient leurs donnees vers une URL CloudFront. CloudFront joue le role de point d'entree mondial (CDN) et transmet les requetes a l'API Gateway, qui declenche la fonction Lambda. La Lambda traite les donnees, les sauvegarde brutes dans S3 et enregistre les metriques agregees dans DynamoDB.

```
[Capteur IoT] --HTTP POST JSON--> [CloudFront]
|                                     [API Gateway] /ingest
|                                     [Lambda Python 3.11]
|                                     [S3 Data Lake] [DynamoDB]
raw-zone/...      metriques
```

Sous-systeme 2 : Documentation technique (accès securise)

Le bucket S3 de documentation est entierement prive (aucun acces public). Seul le CDN CloudFront est autorise a lire son contenu, grace au mecanisme OAC qui signe chaque requete avec la signature AWS SigV4.

```
[Navigateur] --> [CloudFront + OAC] -- SigV4 --> [S3 Bucket prive]
index.html
```

Ressources AWS creees par CloudFormation

Le template CloudFormation definit 13 ressources AWS interconnectees, deployees atomiquement en une seule operation.

No.	Nom dans le template	Service AWS	Description
1	S3BucketRaw	Amazon S3	Stocke les fichiers JSON bruts (Data Lake)
2	S3BucketDoc	Amazon S3	Heberge la documentation HTML (bucket prive)
3	DynamoDBTable	Amazon DynamoDB	Enregistre les metriques agregées par requete
4	LambdaExecutionRole	AWS IAM	Donne a la Lambda le droit d'ecrire dans S3 et DynamoDB
5	LambdaFunction	AWS Lambda	Traite les donnees IoT (Python 3.11)
6	RestApi	API Gateway	Expose le endpoint HTTP POST /ingest
7	IngestMethodPOST	API Gateway	Definit la route POST /ingest
8	ApiDeployment	API Gateway	Publie l'API sur le stage prod
9	LambdaApiGatewayPermission	AWS Lambda	Autorise API Gateway a invoquer la Lambda
10	CloudFrontIngestion	CloudFront	CDN devant l'API Gateway (point d'entree IoT)
11	CloudFrontOAC	CloudFront OAC	Signe les requetes vers S3 avec SigV4
12	CloudFrontDoc	CloudFront	CDN devant le bucket de documentation
13	DocBucketPolicy	S3 Bucket Policy	Autorise uniquement CloudFront a lire le bucket doc

Etape 1 - Preparation de l'environnement

Avant de deployer quoi que ce soit, il faut s'assurer que les outils necessaires sont en place sur votre machine.

1.1 Verifier que l'AWS CLI est installe

L'AWS CLI est l'outil en ligne de commande qui permet d'interagir avec AWS sans passer par la console web.

```
aws --version
```

Si la commande retourne quelque chose comme `aws-cli/2.x.x`, vous etes pret.

1.2 Configurer vos identifiants AWS

Configurez vos cles d'accès AWS. Ces cles se trouvent dans la console AWS sous "Security credentials".

```
aws configure
```

Renseignez les informations suivantes : AWS Access Key ID, AWS Secret Access Key, Default region name (eu-west-3), Default output format (json).

1.3 Verifier que la connexion fonctionne

```
aws sts get-caller-identity
```

1.4 Installer la bibliotheque Python pour les tests

```
pip install requests
```

Etape 2 - Deploiement de l'infrastructure avec AWS SAM CLI

Le template utilise Transform: AWS::Serverless-2016-10-31 et une ressource AWS::Serverless::Function. Ce template ne peut pas etre deploye directement via la console AWS ni via aws cloudformation deploy sans passer par SAM CLI au préalable. SAM est necessaire pour packager le code Python et transformer le template avant envoi a CloudFormation.

2.1 Verifier les pre-requis SAM et Docker

```
sam --version # SAM CLI, version 1.x.x docker --version # Docker est requis pour sam build
```

2.2 Valider le template

```
sam validate --region eu-west-3 --template infrastructure/template.yaml --lint
```

Reponse attendue : infrastructure/template.yaml is a valid SAM Template

2.3 Construire le package Lambda

```
sam build --region eu-west-3 --template infrastructure/template.yaml
```

SAM lit infrastructure/template.yaml, localise le code via CodeUri: ../src/, installe les dependances de src/requirements.txt et genere l'artefact deployable dans .aws-sam/build/.

2.4 Premier deploiement (mode guide interactif)

```
sam deploy --guided \ --region eu-west-3 \ --stack-name tp-final-iot-pipeline-  
stchabodi \ --template-file .aws-sam/build/template.yaml \ --no-fail-on-empty-  
changeset \ --parameter-overrides Environment=stchabodi \ --capabilities  
CAPABILITY_IAM
```

2.5 Suivre la progression du deploiement

Le deploiement prend environ 3 a 5 minutes. CloudFormation cree les ressources dans un ordre precis en respectant les dependances.

2.6 Verification - Pile CloudFormation CREATE_COMPLETE

La capture d'ecran ci-dessous montre l'interface CloudFormation avec la pile tp-final-iot-pipeline-stchabodi au statut CREATE_COMPLETE, confirmant que toutes les ressources ont ete creees avec succes.

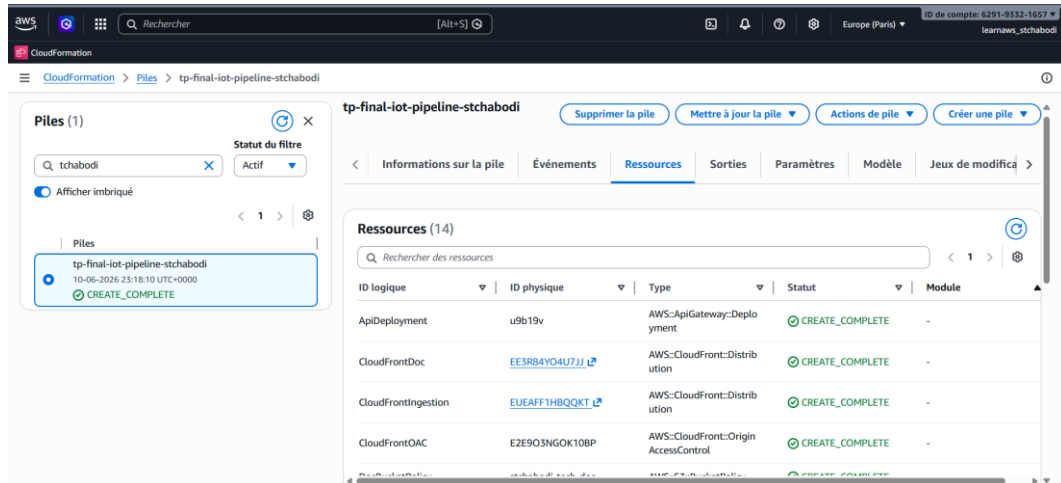


Figure 1 : Pile CloudFormation au statut CREATE_COMPLETE

2.7 Récupérer les URLs générées dans les Outputs

Une fois la pile déployée, CloudFormation expose les URLs importantes dans l'onglet "Sorties". Ces URLs sont générées dynamiquement et propres a votre déploiement.

Notez impérativement ces deux valeurs :

- CloudFrontIngestionURL : l'URL pour envoyer les données IoT (<https://XXXXXXXXXXXXXXXX.cloudfront.net/ingest>)
- CloudFrontDocURL : l'URL pour accéder a la documentation (<https://YYYYYYYYYYYYYYYY.cloudfront.net>)

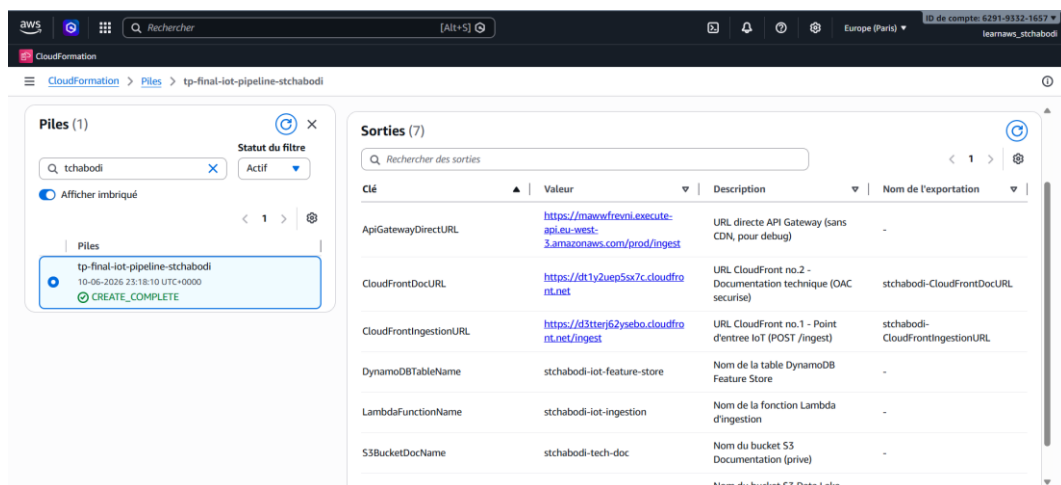


Figure 2 : Onglet Outputs de la pile CloudFormation avec les deux URLs

Etape 3 - Comprendre le code de la fonction Lambda

Le fichier `src/index.py` contient le code complet de la fonction Lambda. L'architecture choisie est FastAPI + Mangum. FastAPI gere le routage HTTP et la validation des données via Pydantic. Mangum est l'adaptateur qui permet d'exécuter une application ASGI/FastAPI dans le runtime AWS Lambda.

```
[API Gateway Event] --> [Mangum adapter] --> [FastAPI app] --> [index.py handlers]
```

3.1 Ce que fait la Lambda, étape par étape

Etape A - Validation automatique du schéma par Pydantic

Pydantic valide automatiquement le corps de la requête contre les modèles déclarés. Si un champ requis (`sensor_id`, `status`) est absent ou si le type est incorrect, FastAPI retourne HTTP 422 sans même entrer dans la fonction handler.

Etape B - Vérification de la liste records

Si `records` est une liste vide, une `ValueError` est levée explicitement.

```
if not records:    raise ValueError("Le champ 'records' est vide - aucun
enregistrement a ingerer")
```

Etape C - Sauvegarde brute dans S3 avec partitionnement temporel

Le payload est sérialisé en JSON et sauvegardé dans S3 avec un chemin structure : `raw-zone/year=YYYY/month=MM/<request_id>.json`

Etape D - Calcul des métriques à la volée

La Lambda calcule la température moyenne, le nombre d'erreurs et le nombre d'enregistrements.

Etape E - Enregistrement dans DynamoDB

Les métriques sont stockées dans DynamoDB avec le `request_id` comme clé primaire.

Etape F - Réponse HTTP 201

FastAPI retourne HTTP 201 avec le resume de l'ingestion.

3.2 Gestion des erreurs

Scenario	Déclencheur	Comportement	Code HTTP
Schema invalide	Pydantic ValidationError	FastAPI retourne 422 automatiquement	422
Records vide []	ValueError explicite	Exception loguee, Mangum retourne 500	500
Erreur AWS	Exception boto3	Exception loguee, Mangum retourne 500	500

Etape 4 - Exécution des tests avec test_client.py

Ouvrez test_client.py dans votre editeur et remplacez la variable CLOUDFRONT_INGESTION_URL par l'URL récupérée dans les Outputs CloudFormation.

```
CLOUDFRONT_INGESTION_URL = "https://d1abc2defghijk.cloudfront.net/ingest"
```

Depuis le dossier devoir_final, exécutez :

```
python test_client.py
```

4.1 Test 1 - Payload valide (HTTP 201 attendu)

Ce test envoie un batch de 4 mesures capteurs bien formées. Deux capteurs sont en statut OK et deux en statut ERROR. Les températures sont 72.5, 89.3, 68.1 et 95.7 degrés C.

Calcul attendu : $(72.5 + 89.3 + 68.1 + 95.7) / 4 = 81.4$ C | Anomalies : 2

Si vous obtenez HTTP 201 avec ces valeurs, le pipeline fonctionne de bout en bout.

4.2 Test 2 - Records vide (provoque un HTTP 500)

Ce test envoie un payload avec records: [] (liste vide). La fonction ingest_iot_data détecte la liste vide et lève un ValueError. Le gestionnaire global FastAPI logue le stack trace complet et relève l'exception.

4.3 Test 3 - Schema invalide : sensor_id manquant (HTTP 422 attendu)

Ce test envoie un enregistrement sans le champ sensor_id qui est requis dans le modèle Pydantic. FastAPI détecte automatiquement la violation de schema et retourne HTTP 422 (Unprocessable Entity).

Etape 5 - Vérification des données dans Amazon S3

Après le Test 1, le fichier JSON brut doit être présent dans le bucket Data Lake.

5.1 Lister les fichiers ingérés via la CLI

```
aws s3 ls s3://stchabodi-iot-datalake/raw-zone/ --recursive --region eu-west-3
```

Vous devriez voir une ligne comme : raw-zone/year=2026/month=06/<request_id>.json

5.2 Vérification depuis la console AWS

Dans la console AWS, accédez au service S3, cliquez sur le bucket stchabodi-iot-datalake, puis naviguez : raw-zone/ -> year=2026/ -> month=06/

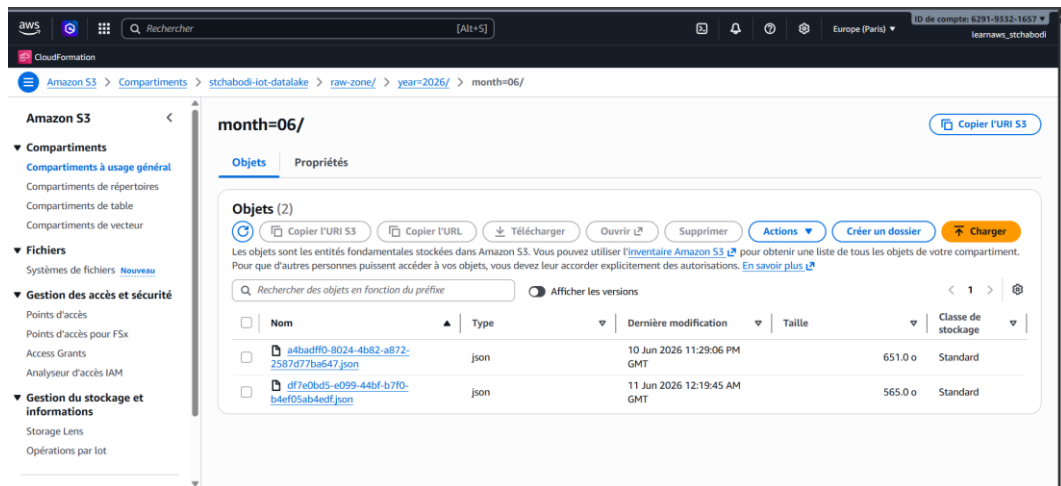


Figure 3 : Arborescence S3 montrant le partitionnement temporel avec un fichier JSON

Etape 6 - Vérification des métriques dans Amazon DynamoDB

6.1 Scanner la table via la CLI

```
aws dynamodb scan \ --table-name stchabodi-iot-feature-store \ --region eu-west-3 \
--output json
```

Chaque element de la reponse correspond a une execution de la Lambda, contenant : request_id, timestamp, s3_path, avg_temperature, error_count et record_count.

6.2 Vérification depuis la console AWS

Dans la console AWS, accédez au service DynamoDB -> "Tables" -> sélectionnez stchabodi-iot-feature-store -> onglet "Explorez les éléments de table".

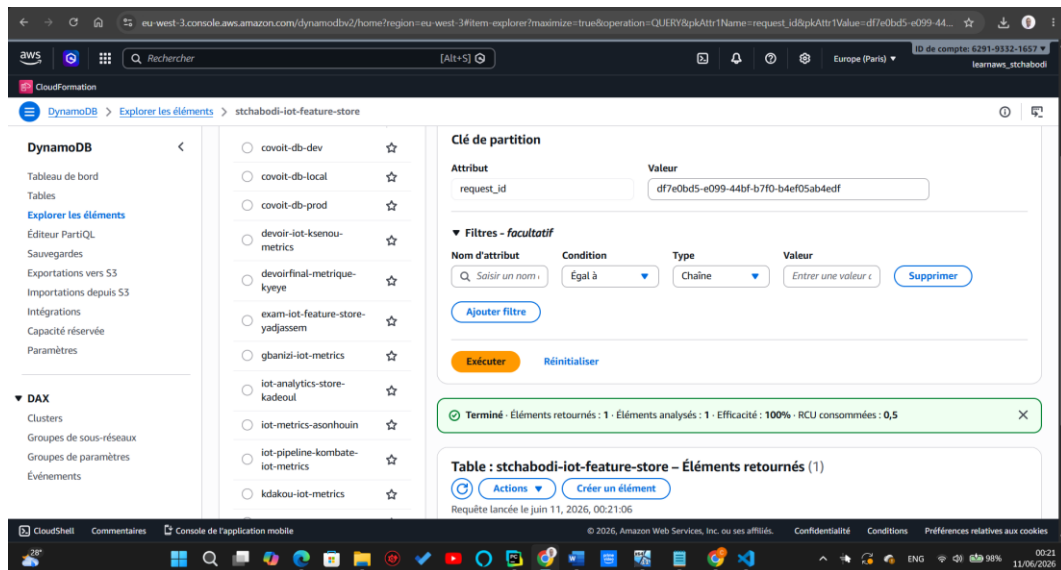


Figure 4 : Table DynamoDB avec une ligne insérée montrant avg_temperature (81.4) et error_count (2)

Etape 7 - Déploiement de la documentation technique

7.1 Uploader le fichier HTML

```
aws s3 cp index.html s3://stchabodi-tech-doc/index.html \ --content-type "text/html; charset=utf-8" \ --region eu-west-3
```

7.2 Tester le blocage de l'accès direct S3

Ouvrez votre navigateur et tentez d'accéder directement au fichier via son URL S3. Vous devez obtenir une page d'erreur Access Denied (HTTP 403). C'est le comportement attendu : le bucket a le Block Public Access active.

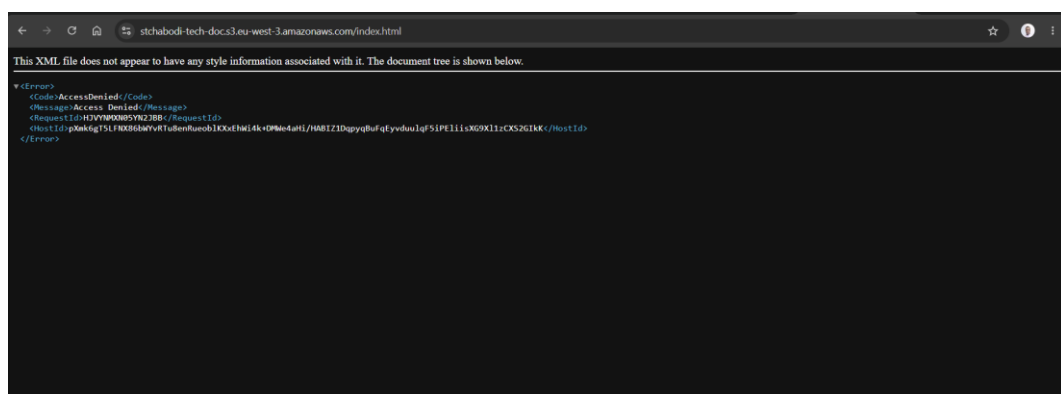


Figure 5 : Message Access Denied lors d'un accès direct au bucket S3

7.3 Accéder via CloudFront OAC (accès sécurisé)

Ouvrez l'URL CloudFrontDocURL recuperee dans les Outputs CloudFormation. La page HTML de documentation s'affiche correctement. CloudFront intercepte la requête, la signe avec SigV4 vers S3, et S3 l'accepte car la Bucket Policy autorise spécifiquement cette distribution CloudFront.

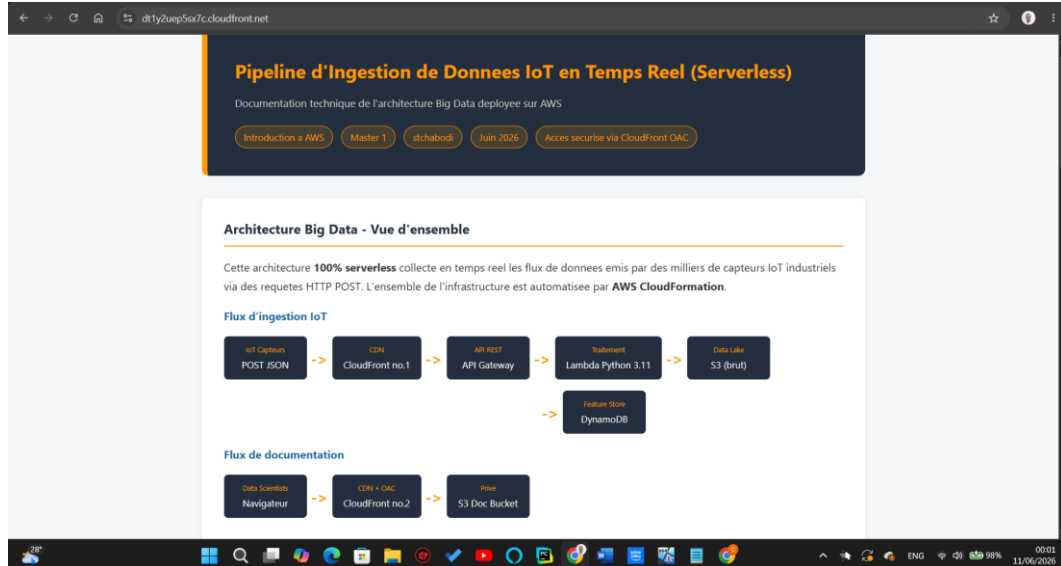


Figure 6 : Page de documentation affichée via l'URL CloudFront sécurisée

Etape 8 - Monitoring et débogage avec Amazon CloudWatch

CloudWatch est l'outil de surveillance d'AWS. Pour une architecture serverless, c'est la seule façon de voir ce qui se passe à l'intérieur des fonctions Lambda. Chaque instruction `print()` dans le code est enregistrée automatiquement.

8.1 Accéder aux logs de la Lambda

Dans la console AWS : recherchez et ouvrez le service "CloudWatch" -> "Journaux" -> "Groupes de journaux" -> cliquez sur `/aws/lambda/stchabodi-iot-ingestion`.

8.2 Log d'une exécution réussie (Test 1)

Après le Test 1, ouvrez le flux de log correspondant. Vous devez trouver cette séquence qui prouve que chaque étape du pipeline s'est bien déroulée :

```
[START] request_id=... | 4 enregistrement(s) [S3] Fichier sauvegarde : s3://stchabodi-iot-datalake/raw-zone/... [METRICS] avg_temperature=81.4C | error_count=2 | record_count=4 [DYNAMODB] Rapport enregistré : request_id=... [END] Succes - HTTP 201
```

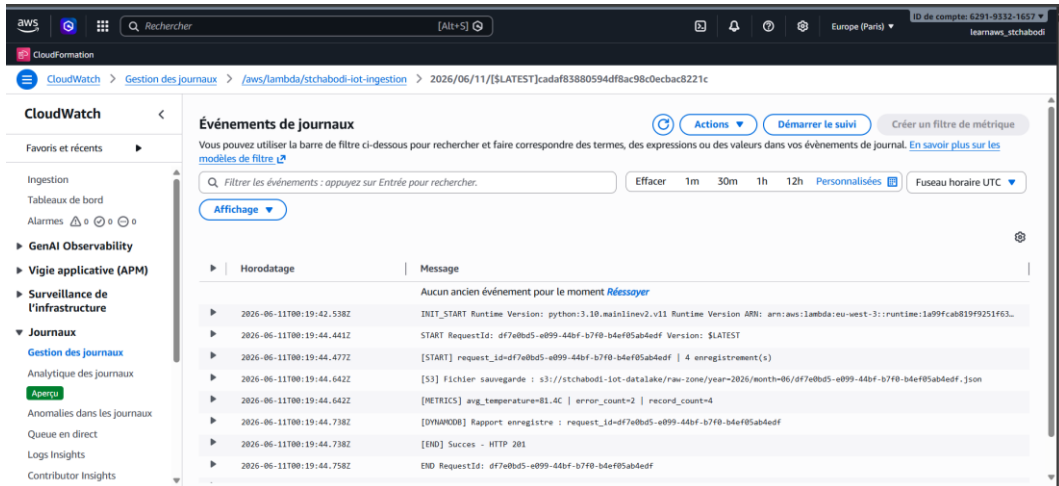


Figure 7 : Log CloudWatch d'une execution reussie (Test 1)

8.3 Log d'une exécution en échec (Test 2 - ValueError)

Le Test 2 a envoyé records: []. L'endpoint FastAPI leve un ValueError. Retrouvez dans CloudWatch le flux de log du Test 2. Vous devez voir le message d'erreur et le Traceback :

```
[ERROR-INTERNAL] ValueError: Le champ 'records' est vide Traceback (most recent call last):
File "/var/task/index.py", line XX, in ingest_iot_data raise
ValueError(...)
```

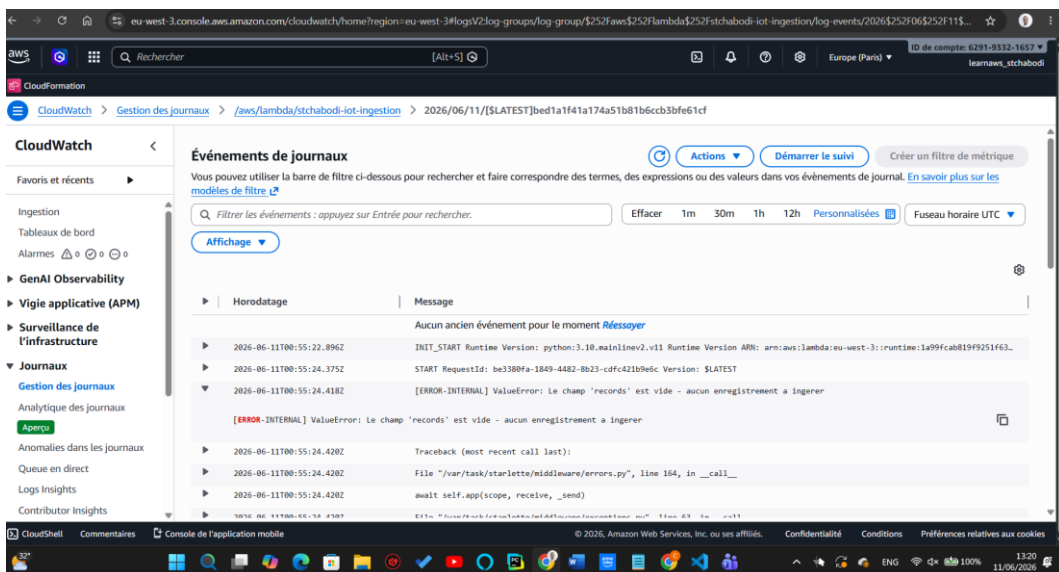


Figure 8 : Log CloudWatch montrant le stack trace Python complet du ValueError (Test 2)

8.4 Comprendre la différence entre les deux types d'erreurs

Erreur de validation schema (Test 3 - Pydantic ValidationError) : FastAPI détecte que le schema du payload n'est pas respecté et retourne automatiquement HTTP 422 sans entrer dans le handler. Le stack trace n'est pas visible dans CloudWatch car l'exception est gérée en interne par FastAPI.

Erreur interne relevee (Test 2 - ValueError sur records vide) : le handler lève un ValueError, le `global_exception_handler` logue le stack trace complet dans CloudWatch puis execute `raise`. Mangum intercepte l'exception et retourne HTTP 500 de façon propre a API Gateway.

Réponses aux questions théoriques

Question 1 - Infrastructure as Code et AWS CloudFormation

L'Infrastructure as Code est une pratique qui consiste à décrire l'infrastructure informatique sous forme de fichiers texte visionnables, exactement comme du code source. Au lieu de cliquer dans une interface graphique pour créer un serveur ou une base de données, on écrit un fichier YAML ou JSON qui décrit ce que l'on veut, et un outil s'occupe de le créer, modifier ou supprimer automatiquement.

Les avantages sont concrets et significatifs dans un contexte professionnel. La reproductibilité permet de déployer exactement la même infrastructure en dev, en staging et en prod avec le même fichier. On élimine les erreurs humaines liées a des configurations manuelles différentes d'un environnement a l'autre. La traçabilité est obtenue car l'infrastructure est dans un fichier texte versionnable avec Git. On sait qui a modifié quoi, quand et pourquoi. L'automatisation est rendue possible car on peut intégrer le déploiement dans un pipeline CI/CD.

AWS CloudFormation est le service IaC natif d'AWS. Il prend un fichier template YAML ou JSON en entrée et gère le cycle de vie complet de l'infrastructure. Lors de la création, CloudFormation analyse les dépendances entre ressources et les crée dans le bon ordre. Lors d'une modification, CloudFormation calcule un change set et n'applique que les changements nécessaires. En cas d'erreur, CloudFormation effectue un rollback automatique.

Question 2 - AWS Lambda et le modèle Serverless vs Amazon EC2

AWS Lambda est un service de calcul qui exécute du code en réponse a des événements, sans gestion de serveur. On fournit le code Python, on configure les permissions et les variables d'environnement, et AWS s'occupe de tout le reste.

Avec EC2, on loue une machine virtuelle. On choisit sa taille, on installe un OS, on configure un serveur web, on gère les mises a jour de sécurité, et on paye l'instance 24h/24 même si elle ne traite aucune requête. C'est l'équipe qui est responsable de maintenir le serveur en fonctionnement.

Avec Lambda, on ne paye qu'au moment où le code s'exécute, a la milliseconde. Quand aucune donnée IoT n'arrive, la Lambda ne tourne pas et ne coûte rien. Dès que des données arrivent, AWS démarre automatiquement autant d'instances que nécessaire.

Aspect	Lambda (Serverless)	EC2 (Instance virtuelle)
Provisionnement	Zero - AWS gere tout	Manuel : type, AMI, reseau
Facturation	A la milliseconde d'exécution	A l'heure, même au repos
Scalabilite	Automatique, instantanée	Auto Scaling Groups a configurer
Maintenance OS	Prise en charge par AWS	A la charge de l'équipe
Duree max	15 minutes	Illimitée
Etat	Stateless	Stateful possible

Pour un use case IoT avec des capteurs qui envoient des données de façon sporadique, Lambda est nettement plus adapte. EC2 serait gaspilleur car l'instance consommerait des ressources même quand aucun capteur ne transmet.

Question 3 - L'interet d'un CDN CloudFront devant l'API Gateway pour l'IoT

Placer CloudFront devant l'API Gateway apporte plusieurs benefices essentiels pour une infrastructure IoT mondiale.

- Latence globale reduite : CloudFront dispose de plus de 400 Points of Presence repartis dans le monde entier. Quand un capteur installe en Asie envoie ses donnees, la requete arrive d'abord au PoP CloudFront le plus proche, qui etablit une connexion optimisee vers notre API en Europe.
- Protection contre les attaques : CloudFront absorbe le trafic malveillant en bordure du reseau, avant qu'il n'atteigne l'infrastructure. On peut egalement coupler CloudFront avec AWS WAF pour filtrer les requetes suspectes.
- Stabilite des endpoints : si on reconstruit l'API Gateway, l'URL CloudFront reste inchangee. Les capteurs IoT deployes sur le terrain n'ont pas besoin d'etre reconfigures.
- Reduction des couts : pour les routes GET, grace au cache CloudFront. Pour nos POST d'ingestion, le cache est desactive (TTL=0) car chaque requete est unique.

Question 4 - S3 comme Data Lake et DynamoDB comme Serving Layer

Dans une architecture Big Data, on distingue deux couches de stockage avec des roles tres differents.

S3 comme Data Lake permet un stockage illimite a faible cout (quelques centimes par Go par mois). Les donnees y sont conservees dans leur format original, sans transformation, ce qui garantit qu'on peut toujours retraiter depuis la source si les besoins analytiques evoluent. AWS Athena peut faire des requetes SQL directement sur les fichiers JSON stockes dans S3, sans serveur. Le partitionnement temporel applique permet a Athena de ne scanner que les partitions pertinentes.

DynamoDB comme Serving Layer garantit une latence inferieure a 10 millisecondes pour les lectures par cle primaire. C'est precisement ce dont ont besoin les applications temps reel. La table

DynamoDB stocke des donnees deja agregees et calculees, directement exploitables par une application de monitoring sans traitements supplementaires.

Une base relationnelle unique (PostgreSQL, MySQL) serait inadaptée car le modele transactionnel est inadapté a l'ingestion de millions de lignes par heure, le stockage est bien plus couteux que S3, et les SGBDR ne scalent pas horizontalement sans ingenierie complexe.

Question 5 - Le modele de responsabilite partagee AWS applique a S3

Le modele de responsabilite partagee definit la frontiere entre ce qu'AWS securise et ce que le client doit securiser.

AWS est responsable de la securite de l'infrastructure physique et logicielle qui fait tourner S3 : les datacenters, les serveurs physiques, les cables reseau, et le logiciel S3 lui-meme. Si AWS a une faille dans son propre logiciel, c'est sa responsabilite.

Le client est responsable de ce qui concerne son utilisation de S3 : les politiques d'accès (Bucket Policy, ACL, IAM), le chiffrement des donnees, la configuration du Block Public Access, et la gestion des identifiants. Si les clés AWS IAM sont compromises, c'est le client qui doit les revoke.

Dans ce projet, nous appliquons ce modele en configurant BlockPublicAccess a true sur le bucket de documentation, une Bucket Policy qui n'autorise que la distribution CloudFront, et un role IAM pour la Lambda avec uniquement les permissions minimales.

Question 6 - Static Website Hosting public vs CloudFront avec OAC

L'option Static Website Hosting de S3 permet d'exposer un bucket comme un site web public. C'est pratique pour des tests, mais totalement inadapté pour une documentation interne confidentielle. Des qu'un bucket est en hebergement public, son URL est accessible par n'importe qui sur Internet sans authentication. Des bots peuvent scanner et indexer le contenu. L'URL S3 utilise HTTP par default.

Notre approche avec CloudFront et OAC est radicalement differente. Le bucket S3 est entierement prive avec toutes les options Block Public Access activees. La distribution CloudFront est configuree avec un Origin Access Control. L'OAC signe chaque requete vers S3 avec AWS Signature V4. La Bucket Policy n'autorise que les requetes signees par cette distribution CloudFront specifique. Quand un utilisateur ouvre l'URL CloudFront, CloudFront recoit la requete via HTTPS, signe une nouvelle requete vers S3 avec SigV4, et S3 la retourne apres verification.

Question 7 - Amazon CloudWatch pour superviser et deboguer une Lambda

Quand on developpe une application web classique, on peut se connecter au serveur et lire les logs. Avec une architecture serverless, la Lambda s'exécute dans un environnement ephemere gere par AWS. CloudWatch est donc l'outil de diagnostic incontournable.

Chaque invocation de la Lambda cree une entree dans CloudWatch Logs. Tout ce que le code ecrit via `print()` est automatiquement enregistre. En plus des logs, CloudWatch collecte des metriques automatiquement : Invocations, Duration, Errors, Throttles. Ces metriques permettent de configurer des CloudWatch Alarms pour envoyer des alertes automatiques.

Quand la Lambda leve une exception non geree, le runtime Lambda la recoit, marque l'invocation comme ERROR, enregistre le Traceback Python complet, et incremente la metrique Errors.

Question 8 - Pourquoi Lambda atteint ses limites avec 50 Go, et quelle alternative

Lambda est excellent pour traiter des evenements de petite taille en temps reel. Mais ses contraintes techniques le rendent inadapte aux gros volumes :

- Duree max d'execution : 15 minutes. Lire 50 Go depuis S3, les parser et les transformer prendrait bien plus de temps.
- Memoire maximale : 10 Go de RAM. Un fichier de 50 Go ne peut pas etre charge en memoire.
- Payload HTTP maximal : 6 Mo via API Gateway. Un fichier de 50 Go ne peut pas etre envoye via une requete HTTP classique.
- Stockage temporaire /tmp : limite a 10 Go.

La solution recommandee est AWS Glue avec Apache Spark. Glue est un service ETL serverless specifiquement concu pour traiter des volumes massifs. Spark distribue le traitement sur plusieurs machines en parallele. Un fichier de 50 Go est automatiquement decoupe en partitions traitees simultanement.

Checklist de validation finale

Infrastructure et deploiement :

- Pile CloudFormation creee avec statut CREATE_COMPLETE
- Les 2 URLs CloudFront recuperees dans les Outputs de la pile
- `test_client.py` configure avec la valeur de CloudFrontIngestionURL

Pipeline IoT (Test 1 - payload valide) :

- HTTP 201 recu en reponse au payload de 4 capteurs
- Fichier JSON present dans S3 sous `raw-zone/year=.../month=.../`

- Entree DynamoDB creee avec request_id, timestamp, s3_path, avg_temperature, error_count

Documentation technique (sous-systeme 2) :

- index.html uploadé dans le bucket -tech-doc via la commande AWS CLI
- Acces direct via URL S3 retourne HTTP 403 Access Denied
- Acces via CloudFrontDocURL affiche la page HTML correctement

Monitoring CloudWatch :

- Log d'une execution reussie visible (Test 1 - sequence START->S3->DYNAMODB->END)
- Stack trace Python complet visible pour l'execution en echec (Test 2 - ValueError)

Preparation du ZIP pour la soumission

Structure attendue dans le ZIP :

```
devoir_final_stchabodi.zip |— infrastructure/ |   └─ template.yaml |— src/ |   └─
index.py |— test_client.py |— index.html └─ rapport_stchabodi.pdf
```

Documentation Technique

Pipeline IoT Serverless AWS

TCHABODI Sadikou | Master 1 IA-BIG DATA | Juin 2026

Cours : Introduction à AWS | Responsable : M. Hervé LOKOSSOU